A Lightweight Framework for Fast Trajectory Simplification

Ziquan Fang†, Changhao He†, Lu Chen†, Danlei Hu†, Qichen Sun†, Linsen Li‡, Yunjun Gao†
†*Zhejiang University, Zhejiang Province, China*
‡*Hikvision Research Institute, Zhejiang Province, China*
†{*zqfang, chehe, luchen, dlhu, sunqichen, gaoyj*}*@zju.edu.cn* ‡*lilinsen@hikvision.com*

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00184

***Abstract*—The ubiquitous GPS sensors collect massive trajectory data from moving objects, which is useful in data mining applications. However, trajectory data is enormous in volume, and thus, directly storing and processing the raw data is expensive. Using *trajectory simplification*, a trajectory can be reduced to a set of continuous line segments with acceptable data loss, which is an efficient method. Although many algorithms are proposed, they still suffer from the following issues including (i) non-data driven capability as most studies rely on human-crafted rules or pre-defined parameters, (ii) bound with error measures that yield high computational cost, and (iii) focusing only on the local information preservation in trajectories, but failing in capturing the global mobility patterns for trajectory compression.**

To address the above issues, we propose a Seq2Seq2Seq framework, abbreviated S3, which consists of two chained Seq2Seq. With differentiable reconstruction learning, S3 enables self-supervised trajectory simplification in a lightweight manner. Besides, we deploy S3 over the graph neural architecture to capture the context-aware mobility patterns and enhance the representation paradigm of trajectories with geographical semantics, where a context-aware distance measure is designed for quality evaluation. An online extension of S3 is also developed to enable streaming trajectory simplifications. Finally, extensive experiments using two real-world datasets in both offline and online scenarios show that S3 achieves much higher efficiency (e.g., it achieves up to one order of magnitude speed-up gains) and comparable compression quality, compared with both non-learning and state-of-the-art learning-based methods.

***Index Terms*—Trajectory simplification, deep representation learning, sequence-to-sequence**

I. Introduction

With the development of techniques for positioning, GPS sensors with high sampling rates and low prices are widely used in smartphones, vehicles, and wearable devices. These built-in GPS sensors produce massive trajectory data that record the traces of moving objects (e.g., pedestrians and vehicles). For example, the world's largest ride-sharing company Uber hits 10 billion rides in July, 2018 [1]. Based on the collected trajectory data, many trajectory data mining tasks [41], [64] such as similarity queries, clustering analytics, and traffic prediction are studied, benefiting a broad range of applications in traffic management [20], urban computing [65], and behavior study [7]. Since trajectories capture the hidden mobility patterns and complex interactions with the located context of moving objects, driven by the success of machine learning, increasing AI-based trajectory data analytics algorithms [53] are proposed recently, yielding similarity learning [28], deep clustering [11], and deep prediction [13]. The community has reached a point where more and more researchers embrace AI techniques as an integral part of trajectory analytics.

The volume of trajectory data is continuously growing in terms of two aspects: i) higher sampling rates of sensors enable acquiring more fine-grained locations of each moving object, which leads to longer trajectories, e.g., a single trajectory may contain hundreds to thousands of time-stamped GPS points; and ii) more monitored moving objects (e.g., city-wide taxis) generate trajectory data simultaneously. Directly transmitting and storing such massive data brings pressure on network bandwidth and storage capacity, which also makes trajectory mining processing expensive. Recall that redundancy commonly exists in trajectories. For instance, all sampling points but the first and last could be dropped when a vehicle travels in a straight road segment at the same speed. Hence, the community calls for trajectory compression [4], [9], [19], [36], [37], [42], [54], [55] to reduce data volumes, which serves as the cornerstone of downstream trajectory analyses.

Lossless compression is widely studied for text and images, but it has been proved to be ineffective (low compression ratios) for trajectory data, according to the experiments of previous work [8], [18], [45]. Basically, a reasonable approximation of query results is acceptable in most real-life applications such as hot area detection and similarity queries [14], [39]. Correspondingly, a tolerable information loss of the original trajectory is acceptable, as long as the query results based on that are not deviated too far. Thus, many research efforts have been devoted to lossy compression. Trajectory simplification, which preserves the remaining points of a simplified trajectory while dropping some points from a raw trajectory, is now the most common lossy compression method, offering a much higher compression ratio with acceptable data loss. Existing trajectory simplification algorithms fall into two categories: (i) min-#: it generates a simplified trajectory with errors bounded in ϵ and as few as trajectory points based on a raw trajectory with n points and (ii) min-ϵ: given a trajectory with n points and a positive integer m, it finds a subsequence consisting of m points to approximate this trajectory and attempts to minimize the compression error. Specifically, we study min-ϵ oriented trajectory simplification as it is convenient to control the compression ratio (i.e., $\frac{m}{n}$) and especially effective for resource-constrained sensors [31], [62].

Quite a few algorithms exist for min-ϵ trajectory simplification. In the early stage, optimal (or called exact) algorithms such as Bellman [4] and Error-Search [37] are proposed. The exact algorithms typically have a high complexity, making them impractical for large-scale trajectory analytics. As a result, approximate algorithms are more preferred, which can

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00184

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TIME OF COMPRESSING A TRAJECTORY USING THE DAD METRIC

Methods	Error-Search [37]	Bottom-Up [36]	Top-Down [19]	RLTS [54]
GeoLife	10,000+s	28.91s	16.74s	5.34s
T-drive	10,000+s	22.71s	23.39s	4.88s

be classified into two categories:

(i) Non-learning-based methods include popular DP [9], Top-Down [19], Bottom-Up [36], etc. All these methods first select an existing error metric such as PED, SED, and DAD (see Sec. III) or they directly define a new error metric [29], [36]. They compute the "importance value" of each point to capture the compression error that results from dropping this point, guided by the chosen error metric. The point with the lowest importance value is then discarded. Last but not least, the remaining points' importance values are updated using a variety of techniques that are already in use.

(ii) Learning-based methods. Even though the methods that are not based on learning have performed well, the majority of them utilize human-made rules to decide which points to simplify. However, trajectories exhibit diverse dependencies such as varying samplings and geo-metrics. These manually designed algorithms are hard to adapt to different scenarios [54]. Recently, as AI has made great success in many fields, there is a trend of designing AI-based trajectory analysis solutions [53] such as trajectory embedding [28], similarity learning [12], [17], [59], [63], deep trajectory clustering [11], [60], to name but a few. Following this trend, Wang et al. [54], [55] propose a reinforcement learning-based method, called RLTS, the first and only existing AI-based approach for trajectory simplification. To learn which point should be discarded, they adapt reinforcement learning and treat trajectory simplification as a MDP problem, thus achieving fewer errors than previous non-learning-based methods.

Motivation. Trajectory simplification solutions are required to be both effective and efficient. According to previous studies [54], [62], both non-learning and learning based methods have achieved high and comparable trajectory simplification quality, i.e., they are effective. However, they face the problem of low efficiency. In terms of non-learning-based methods, they are closely coupled with distance-based error metrics, resulting in computationally intensive and inefficient. In terms of learning-based methods, although RLTS achieves better compression quality, it actually trades efficiency for effectiveness. Specifically, it needs to process a huge state space and massive actions with transitions, bringing computational overhead.As reported in Table I, given the compression ratio 0.1, the running time of compressing a trajectory (selected in the evaluated GeoLife and T-drive datasets) with 450 points (much shorter than the average trajectory length) via existing methods is still high: seconds, dozens of seconds, tens of thousands of seconds. Considering massive trajectories in a dataset required to be compressed, the processing time will increase linearly. Moreover, according to our experimental study, RLTS needs a lot of time for model training before performing one simplification request. Such expensive time costs restrain existing methods from time-aware applications

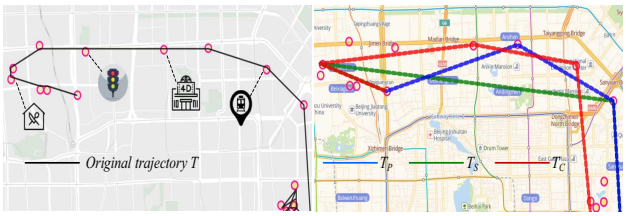


Fig. 1. T denotes the raw trajectory containing semantic-aware POIs; T_P and T_S are T's simplified trajectories using DP approach guided by PED and SED; and T_C is the simplified trajectory using S3 guided by CED.

especially for online or real-time trajectory simplification tasks. In this paper, we propose S3, a lightweight framework, that achieves higher simplification efficiency and comparable effectiveness, compared with existing methods.

Another limitation of existing methods is, they typically focus on local characteristic preservation while ignoring the context-aware global mobility. The local mobility dependence considers the features of intra-trajectory such as position information [44] and direction information [36]. However, the local dependence cannot effectively capture the interaction between trajectories and its located environment, i.e., the global mobility dependence. As shown in Fig. 1 (left), there are some semantic-aware POIs in a long trajectory that should be extracted to decide its context-aware moving trend. The non-learning DP algorithm guided by existing error metrics generates different results, i.e., T_P and T_S, as each metric considers a different perspective of T. More importantly, those error metrics focus on local spatial information preservation, resulting in coarse resolution with global mobility trend loss. In contrast, S3 can identify the POI points during trajectory simplification process. Notice that, there is another bunch of studies called semantic trajectory compression, which leverages external activities or hand-extracted features to facilitate trajectory compression by activity/map matching [14], [15] or clustering [35]. It is not the scope of this paper, as the activity/road features are not always available.

In contrast to previous studies that rely on manual rules or heavyweight reinforcement learning, in this paper, we utilize the lightweight sequence representation learning for trajectory simplification, which means that it chooses which point to drop based on its learned weight during representation learning. This is the first proposal that studies trajectory simplification that trains lightweight neural networks and performs end-to-end trajectory simplification, to the best of our knowledge. However, incorporating representation learning into the trajectory simplification task is not an easy task. Existing deep learning models rely on sufficient labels in a supervised manner. In trajectory simplification, practical scenarios may involve a very limited amount of simplified trajectories due to expensive human efforts. Besides, different error metrics also result in different simplified results (see Fig. 1), i.e., it is impossible to collect ground truth for trajectory simplification.

We propose a self-supervised trajectory simplification framework, abbreviated S3, to address the limitations of existing methods and the aforementioned difficulties. The input

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.

and reconstructed trajectories are represented by the first and last Sequences in S3, while the simplified trajectories are represented by the middle Sequence. As shown in Fig. 2, S3 consists of a Compressor and a Constructor, both of which comprise a Seq2Seq structure. The Compressor receives a raw trajectory T as input, and outputs a simplified trajectory T', which is denoted by ① → ②. In contrast, the Constructor tries to recover the raw trajectory $\hat{T}$ from the simplified trajectory T', which is denoted by ② → ③. We bridge Compressor and Constructor via the reconstruction loss $L_R(T, \hat{T})$, i.e., the Compressor and Constructor communicate through T' while trying to minimize $L_R(T, \hat{T})$, which is denoted by ③ → ①. During model training, S3 distinguishes important points from T to generate T', which corresponds to trajectory simplification. The rationale behind is, the faster for S3 converge, the easier for Constructor to recover $\hat{T}$ based on T', which means the more relatively important points are selected from T by Compressor to form T'. Overall, S3 works effectively for trajectory simplification task.

More importantly, S3 requires no pre-collected ground-truth or error metrics to guide the simplification process. At the same time, trajectory simplification is decoupled from massive error computations, enabling fast processing. We create a context-aware graph neural network that is integrated into S3 and serves as a prior over the latent sequences in order to preserve the context-aware global mobility data and improve the efficiency of compressed trajectories. Besides, we present a new error distance for quality evaluation. Last but not the least, owning to the lightweight architecture of S3, we extend S3 to enable online trajectory simplification processing.

In conclusion, the following contributions are made.

- We present the first deep learning based framework for fast trajectory simplification. The proposed S3 i) is data-driven, ii) runs in a lightweight way without the supervision of labeled data or error computation, and iii) achieves much better efficiency and competitive effectiveness.
- We design a Sequence-to-Sequence-to-Sequence architecture that consists of a Compressor and a Constructor, enabling fast trajectory simplification in a self-supervised manner. A context-aware GNN is further build to preserve both local and global mobility dependencies.
- We extend S3 for streaming settings, so that online trajectory simplification tasks can be also supported.
- We compare S3 with both non-learning and learning based baselines in offline and online modes, and the results show that S3 is much more efficient (e.g., it achieves up to one order of magnitude speed up gains) and also effective (e.g., it yields competitive quality).

II. Related Work

We briefly review offline/online trajectory simplification, road network/semantic-based trajectory compression, and representation learning-based trajectory analytics.

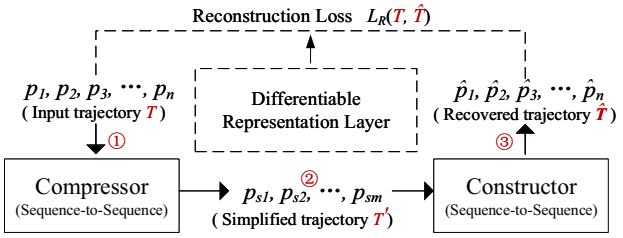


Fig. 2. The architecture of S3

A. Offline and Online Trajectory Simplification

Trajectory simplification is inspired by graphic geometry [56], which roughly fall into two categories: offline and online approaches to trajectory simplification.

In offline settings, the collected trajectories are static and completely available. We only review algorithms about min-ϵ, while the min-# studies [25], [30], [32]–[34], [36], [44] are omitted since they are not the scope of this paper. The Bellman [4] approximation is regarded as the first algorithm for simplifying a trajectory. It aims to generate a subsequence, which has m points with the smallest error, using dynamic programming. The exact algorithm has a high complexity $O(n^3)$. Then, Meratnia et al. [9] introduce the classic Douglas-Peucker (DP) algorithm in a Top-Down manner [19]. $O(n^2)$ is the worst Top-Down time complexity. To improve the efficiency, another implementation of DP called Bottom-Up [36] is proposed. Only the positional information of trajectories is saved by aforementioned methods. To preserve direction information of trajectories, Long et al. [37] develop an exact algorithm (called Error-Search) and an approximate algorithm (called Span-Search), both of which use their specified DAD (direction-aware error metric). When deciding which point to drop, all of these algorithms rely on human-made rules. As AI techniques are propagated in trajectory analyses, Wang et al. [54] use reinforcement learning to develop the first learning-based trajectory simplification. Despite their effectiveness, reinforcement learning brings expensive training costs, highly constraining their deployments. *In contrast, S3 is data-driven but is more lightweight that utilizes self-supervised learning requiring no rules or massive computations, enabling fast trajectory simplification.*

Remote sensors continuously collect and store streaming trajectory data in a buffer (also known as a mini-batch) in online environments. Whenever the buffer becomes full, online trajectory simplification decides which points to drop. Based on that, a large number of methods are proposed in the literature [22], [26], [30], [32], [33], [42]–[44], [47], [52]. On the trajectory data streams, these two methods are local optimization techniques. Their main advantage is their high efficiency, but the quality of their simplifications may decrease significantly [62]. Again, the majority of these strategies are based on rules made by humans. Wang et al. [55] utilize reinforcement learning in an online setting. *In this paper, we are motivated to solve the offline and online min-ϵ trajectory simplification tasks without hurting simplification quality.*

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SYMBOLS AND DESCRIPTION

Notation	Description
p, T	A point, a trajectory $T = \langle p_1, ..., p_n \rangle$ of length n
$T[i:j]$	A sub-trajectory $T[i:j] = \langle p_i, p_{i+1}, ..., p_j \rangle$
$T[s_i:s_j]$	A sub-sequence $T[s_i:s_j] = \langle p_{s_i}, p_{s_{i+1}}, ..., p_{s_j} \rangle$
$\overrightarrow{p_s p_e}$	A directed segment starting from p_s to p_e
T'	A simplified trajectory of T, $T' = \langle p_{s_1}, p_{s_2}, ...p_{s_m} \rangle$
g_t, h_t, v_T	Grid embedding, hidden state, trajectory embedding
$\mathcal{G}$	A context-aware spatial graph
CED	Context-aware error distance function
LR	The reconstruction loss function
$\mathbf{A}$, $\mathbf{H}$, $\mathbf{X}$	Adjacency matrix, Output matrix, Feature matrix

B. Other Types of Trajectory Compression

The vehicle's trajectory is restricted by a road network, the road network space is utilized rather than free space. Thus, [5], [21], [23], [27], [51] use road network's structural characteristics to simplify the trajectory. Since any map-matching algorithm can map a local sequence of successive points to the same road segment, road network-based trajectory simplification reduces spatial redundancy. Frequent travel path mining of people and vehicles can be utilized to reduce the data volumes. However, the above methods are low efficiency due to the map-matching algorithm. There is also another line of studies called semantic trajectory simplification, which replaces original GPS trajectories with semantic-based trajectories, where the semantic means activities [48], stop points [35] or other events [58]. As summarized in [62], in that case, It deletes all points in the moving state and only keeps the location points that represent semantic meanings. As a result, accuracy in simplification cannot be guaranteed. For the two reasons listed below, the road network or semantic-based trajectory compression algorithms are outside the scope. First, additional map-matching or semantic knowledge is required for these methods, which is not always available. [54], [62]. Second, rather than using the original trajectory, these techniques are applied on top of transformed trajectories like road segments or sequential context. *In this paper, we focus on free space, we concentrate on simplifying trajectory simplification for people, vehicles, animals, etc., which has been the subject of a lot of research in the literature.*

C. Deep Representation Learning

Recently, many researchers start leveraging AI techniques to revisit previous non-learning trajectory analytics. Wang et al. provide a comprehensive survey [53]. Basically, representation learning that aims to capture the hidden features in trajectories has been widely studied. Li et al. [28] design an encoder-decoder to obtain hidden trajectory representations. Yao et al. [60] and Fang et al. [11] use representation learning for deep clustering. Other researchers [12], [17], [59] employ neural networks to learn the representation of trajectories and perform Top-k similarity queries based on the embedding vectors. Among these learning-based trajectory analytics, representation learning serves a foundational role. Recall that trajectory simplification is to drop relatively less important points while keeping relatively more important points. Inspired by that, for the first time, we utilize representation learning to enable data-driven trajectory simplification. *In contrast to previous trajectory learning studies that rely on a plenty of supervision labels, we develop a Sequence-to-Sequence-to-Sequence framework (S3), which performs self-supervised trajectory simplification in a lightweight manner.*

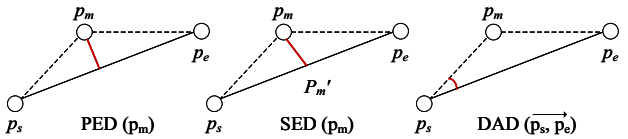


Fig. 3. Illustration of PED, SED, and DAD

III. PROBLEM STATEMENT

The frequently used symbols are summarized in Table II.

Definition 1: **(Point)** *A GPS point p is denoted as a triple (x, y, t) in free space, meaning that a moving object (e.g., a person) is located at 2-dimensional position (x, y) at time t.*

Definition 2: **(Trajectory)** *A trajectory T is a time-ordered sequence of GPS points, i.e., $T_{id} = \langle p_1, p_2, ..., p_n \rangle$, where the subscript id denotes the identifiable id of a moving object and $p_i.t < p_{i+1}.t < p_{i+2}.t < ..., < p_n.t$ ($i \geq 1$)*

Definition 3: **(Sub-trajectory/Sub-sequence)** *A sub-trajectory of T is defined by $T[i:j]$, which ends at point p_j and starts form point p_i: $T[i:j] = \langle p_i, p_{i+1}, ..., p_j \rangle$. A sub-sequence of T is denoted as $T[s_i:s_j] = \langle p_{s_i}, p_{s_{i+1}}, ..., p_{s_j} \rangle$, where $1 \leq s_i < s_{i+1} < ... < s_j \leq n$.*

As illustrated in Fig. 3, sub-trajectory $T[1:3]$ is $\langle p_1, p_2, p_3 \rangle$, while sub-sequence $T[s_1 : s_2]$ is $\langle p_{s_1}, p_{s_2} \rangle = \langle p_1, p_3 \rangle$. Note that, sub-sequence contains non-consecutive points in the original trajectory, while sub-trajectory contains consecutive points. We use $\overrightarrow{p_s p_e}$ as a directed line-segment, pointing from GPS point p_s to GPS point p_e. For simplicity, we compute its angle $\theta\left(\overrightarrow{p_s p_e}\right)$ from the positive x-axis to the vector $\overrightarrow{p_s p_e}$.

Definition 4: **(Anchor Segment)** *The segment $\overrightarrow{p_{s_i} p_{s_{i+1}}}$ is an anchor segment for any points between p_{s_i} and $p_{s_{i+1}}$ in the original trajectory.*

As shown in Fig. 3, $\overrightarrow{p_{s_1} p_{s_2}} (= \overrightarrow{p_1 p_3})$ is the anchor segment for p_2. A sub-sequence with m points and $(m \ll n)$ is basically what is found when trajectory simplification attempts to approximate the initial trajectory T. Specifically, these m points divide the entire trajectory's interval $[1:n]$ into discrete intervals of $m+1$. Data information will be lost if points within an anchor's interval are discarded. loss is measured by measuring the distance between these discarded points and their associated anchor segments.Following previous remarkable studies [31], [32], [44], [51], [62], we select three popular error metrics in the literature: PED, SED, and DAD. In this paper, we also propose a personalized error-distance called context-aware error distance (CED, see Section V-C) by preserving both spatial-aware and context-aware mobility information. Refer to [62], we detail how to compute the introduced information loss when dropping p_m that is assumed to be dropped w.r.t. $\overrightarrow{p_s p_e}$, using PED, SED, and DAD, while CED will be introduced in Sec. V.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.

① **Perpendicular Euclidean Distance (PED).** The PED computes the shortest spatial distance between point p_m and its anchor line-segment $\overrightarrow{p_s p_e}$, which is typically defined as $PED\left(p_m\right) = |\left(y_e - y_s\right) \cdot x_m - \left(x_e - x_s\right) \cdot y_m + x_e \cdot y_s - y_e \cdot x_s| / \sqrt{\left(y_e - y_s\right)^2 + \left(x_e - x_s\right)^2}$.

② **Synchronized Euclidean Distance (SED).** Assuming that the object is moving at a constant speed, the distance is proportional to time, so the distance between projection points is determined by time. Formally, the SED computes the distance between the actual location p_m and its synchronized point p'_m, i.e., $SED\left(p_m\right) = \sqrt{\left(x_m - x'_m\right)^2 + \left(y_m - y'_m\right)^2}$.

③ **Direction-Aware Distance (DAD).** The DAD targets preserving the direction information of moving trajectories. Specifically, when dropping point p_m, the DAD computes the angular difference between $\overrightarrow{p_m p_{m+1}}$ and $\overrightarrow{p_s p_e}$, i.e., $DAD\left(\overrightarrow{p_s p_e}\right) = \max_{s \le m < e} \triangle\left(\theta\left(\overrightarrow{p_s p_e}\right), \theta\left(\overrightarrow{p_m p_{m+1}}\right)\right)$.

PED is efficient for computing, but the speed information of trajectories is lost. SED can obtain the speed (or temporal) information, but introduce more computational costs. DAD is a direction-aware error distance, but cannot accurately evaluate the spatial distance between two trajectories. Nevertheless, all of PED, SED, and DAD are spatial-aware error distances, and they cannot capture the semantic information (e.g., POIs highlighted in Fig. 1 (left)). More details of these metrics can refer to [62]. Fig. 3 illustrates an example, where $\overrightarrow{p_s p_e}$ is the anchor segment for the discarded point p_m. Accordingly, $PED(p_m)$ computes the shortest distance from p_m to the segment $\overrightarrow{p_s p_e}$; $SED(p_m)$ assumes that the object is moving at a constant speed from p_s to p_e and p'_m is the synchronized point of p_m and then $SED(p_m)$ computes the Education distance between p_m and p'_m; $DAD(\overrightarrow{p_s p_e})$ computes the angle between $\overrightarrow{p_s p_m}$ and $\overrightarrow{p_s p_e}$. Refer to previous studies [54], [62], we give the formal problem definition.

Definition 5: **(min-ϵ Trajectory Simplification)** *Given an integer m and a trajectory $T = \langle p_1, p_2, ...p_n \rangle$, min-$\epsilon$ trajectory simplification is to generate a simplified trajectory $T' = \langle p_{s_1}, p_{s_2}, ..., p_{s_m} \rangle$ where $m \le n$ and $1 = s_1 < s_2 < ... < s_m = n$ such that $\epsilon(T')$ (computed by existing or self-designing error metrics) is minimized as much as possible.*

IV. Basic S3 Approach

The rationale behind min-ϵ is that important points in this trajectory typically contribute more to both the representation and reconstruction process for neural network training.

As depicted in Fig. 4, S3 consists of four modules, i.e., Embedding layer, Compressor, Constructor, and Differential layer, highlighted by different colors. The Embedding layer transforms raw data (i.e., positions) into discrete tokens (i.e., vectors) for neural networks, i.e., $p_i \rightarrow g_i$, as neural networks only adapt vectors as inputs. The Compressor takes a trajectory T as an input and produces a simplified trajectory T', while the Constructor tries to recover the input as $\hat{T}$ from the simplified trajectory T'. The Compressor and Constructor comprise an attentional Seq2Seq model each (i.e., encoder-decoder), and communicate in the Differential layer through T'.

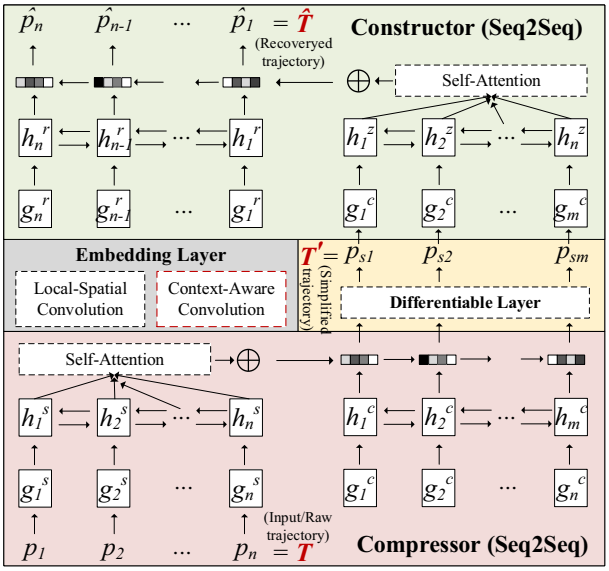


Fig. 4. An overview of S3 consisting of four modules

Consider an original trajectory, e.g., $T = \langle p_1, p_2, ...p_n \rangle$, the Embedding layer projects source sequence T to trajectory embedding $v_T = \langle g_1, g_2, ...g_n \rangle$, which is then feed as the input for Compressor. Based on Seq2Seq (i.e., encoder-decoder), the Compressor takes v_T as an input, and generates a simplified trajectory T', i.e., $v_T \rightarrow T' = \langle p_{s_1}, p_{s_2}, ..., p_{s_m} \rangle$ ($m \le n$ and $1 = s_1 < s_2 < ... < s_m = n$). Next, the Constructor takes T' as an input, and tries to recover the input trajectory $\hat{T}$, i.e., $T' \rightarrow \hat{T} = \langle \hat{p_1}, ..., \hat{p_{n-1}}, \hat{p_n} \rangle$. During forward-backward neural network training, the Compressor ($T \longrightarrow T'$) and Constructor ($T' \rightarrow \hat{T}$) communicate through T', i.e., $T \leftrightarrow T' \leftrightarrow \hat{T}$, which is guided by the reconstruction loss $L_R(T, \hat{T})$. That is, L_R tries to minimize data loss between T and $\hat{T}$, which enables S3 to capture relatively important points from both T and $\hat{T}$ to form the simplified trajectory T'.

A. Embedding Layer (Position Embedding)

Since neural networks can only adopt vectors as input, given a trajectory T consist of real GPS points, we need to transform it into discrete tokens or called vectors. Here, we use the grid partitioning method, which is a common practice to divide the entire area into disjointed grids/cells. All of these grids make up V, which is similar to vocabulary in NLP. A cell is given a vocabulary *id* and treated as a token or word. Then, we use the skip-gram algorithm to initialize the representation of each cell by maximizing $\frac{1}{N} \sum_{t=1}^{N} \sum_{-u \le j \le u, j \ne 0} \log P\left(g_{t+j} \mid g_t\right)$, where the current cell is represented by g_t, a neighbor cell by g_{t+j}, the number of neighbor cells by u, and the total number of cells by N. The following softmax function can be used to calculate the probability because two adjacent cells should have similar representations:

$$\mathbb{P}\left(g_{t+j} \mid g_t\right) = \frac{\exp\left(v_{g_{t+j}}^T v_{g_t}\right)}{\sum_{g_i \in V} \exp\left(v_{g_t}^T v_{g_t}\right)} \tag{1}$$

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.

where the vector representation of g_t, is represented by v_{g_t}, is simply abbreviated as g_t in the next of this paper. As a result, given a raw trajectory T, we can transform it into a spatial embedding v_T: $T \rightarrow v_T = \langle g_1, \cdots, g_t, \cdots, g_{|T|} \rangle$.

B. Compressor

Fig. 4 (the bottom part) illustrates the internals of the Compressor. The input trajectory is first projected by the embedding layer. $T = \langle p_1, p_2, ..., p_n \rangle$ of n points to deep embeddings $v_T = \langle g_1, g_2, ..., g_n \rangle$, which are then feed into the Compressor, generating a simplified trajectory $T' = \langle p_{s_1}, p_{s_2}, ..., p_{s_m} \rangle$ of m points where $m \ll n$. Next, we first give prior knowledge about the Seq2Seq, and then, we detail the designed parallel-attentive encoder-decoder used in S3.

1) Sequence-to-Sequence: A lot of works use the Seq2Seq framework to find the hidden information in sequential data like time series and text. In particular, Seq2Seq is a conditional language model that aims to maximize the probability of the subsequent target word if the input sequence and preamble are known. Let's say that there are currently two sequences of length x and y. Seq2Seq's objective function is as follows:

$$\mathbb{P}(y \mid x) = \mathbb{P}(y_1 \mid x) \prod_{t=2}^{|y|} \mathbb{P}(y_t \mid y_{1:t-1}, x) \tag{2}$$

Seq2Seq typically makes use of an encoder-decoder framework that reads the input sequence x and outputs the sequence y. The lengths of x and y are respectively indefinite. The sequential information in x is encoded by the encoder into a fixed-dimensional vector v, which is then used by the decoder to decode y using the encoded representation v. When training Seq2Seq, a special token (EOS) is used to indicate the end of a sequence so that the Seq2Seq (i.e., encoder-decoder) can be generalized to different settings of input/output lengths. Because it can easily handle input/output sequences of varying lengths, the encoder-decoder architecture is widely used in NLP tasks such as machine translation [24] and sentence compression [10]. The encoder reads the input sequence x sequentially, and the hidden states of RNNs are updated: $h_t^e = \boldsymbol{f}\left(h_{t-1}^e, x_t\right)$, where $1 \leq t \leq |x|$, $\boldsymbol{f}(\cdot, \cdot)$ denote the forward computation of RNNs. In the hidden state, the superscripts e and d are used to differentiate the encoding and decoding phases. After it processed the final token x_t, we get the final hidden state h_t. We then use v_x to preserve the sequential information of input x. As a result, the encoder is able to embed sequential patterns of input sequences thanks to the final hidden state h_t, which actually preserves the internal information at step t. The decoder also typically utilizes RNN-type networks to predict the current output symbol y_t through the current state h_t^d, $h_t^d = f\left(h_{t-1}^d, y_{t-1}, v_x\right)$. Overall, we can rewrite the conditional probability (Eq. 2) to:

$$\begin{aligned} \mathbb{P}(y_t \mid y_{1:t-1}, x) &= \mathbb{P}(y_t \mid y_{1:t-1}, v_x) = \mathbb{P}\left(y_t \mid h_t^d\right) \\ \mathbb{P}\left(y_t = j \mid h_t^d\right) &= \frac{\exp\left(W_j^T h_t^d\right)}{\sum_{j' \in V} \exp\left(W_{j'}^T h_t\right)} \end{aligned} \tag{3}$$

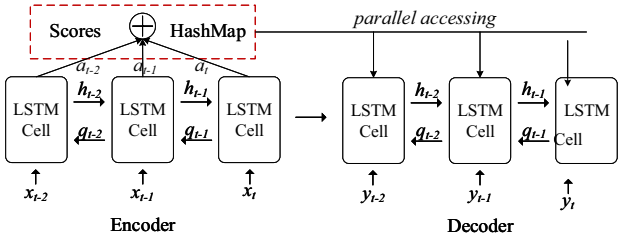


Fig. 5. Illustration of Parallel-Attentive Encoder-Decoder

where j indicates a probable value of V, and h_t^d is projected from the hidden state of the vocabulary via matrix W_j^T

2) Parallel-Attentive Encoder-Decoder: Specifically, both the encoder and decoder use a bidirectional RNN. As depicted in Fig. 4, given v_T, we first encode v_T to produce $h^s = \langle h_1^s, h_2^s, ..., h_n^s \rangle$. Each h_t^s is the concatenation of the Bi-directional hidden states of the Bi-RNNs.

$$h_t^s = [\overrightarrow{RNN}_s(g_t, \overrightarrow{h}^s_{t-1}); \overleftarrow{RNN}_s(g_t, \overleftarrow{h}^s_{t+1})] \tag{4}$$

Self-Attention. To generate the simplified trajectory T', we use the self-attention mechanism, which is a common technique in sequence representation learning studies. Specifically, at each step $t \in \{1, ..., m\}$, To generate a context-aware vector c_t, we use a probability distribution a_i conditioned on the current state h_t^c of the compressor's decoder over all states h_1^s,..., h_n^s of the source encoder.

$$a_i = \mathit{Softmax}(h_i^s W_a h_t^c), c_t = \sum_{i=1}^{n} a_i h_i^s \tag{5}$$

W_a is a parameter matrix. Combining c_t and the decoder's current state h_t^c yields a probability distribution for p_s^t:

$$\begin{aligned} o_t^c &= \mathit{tanh}(W_o[c_t; h_t^c] + b_o), u_t^c = W_v o_t^c + b_v \\ p(y_t | y \leq t, x) &= \mathit{Softmanx}(u_t^c) \end{aligned} \tag{6}$$

W_o, b_o, W_v, and b_v are hyperparameters. The decoder's embedding e_t^c, a countdown argument $m-t$ (scaled by a learnable w_d) indicating the number of the simplified trajectory's remaining points, and the decoder's h_t^c are all updated using the c_t.

$$h_{t+1}^c = \overrightarrow{RNN}_c(h_t^c, g_t^c, c_t, w_d(m-t)) \tag{7}$$

By sampling from a uniform distribution, we obtain a target length m for each input $T = \langle p_1, ..., p_n \rangle$ for the simplified trajectory $T' = \langle p_{s_1}, ..., p_{s_m} \rangle$. Because it is straightforward and effective, we will use the most typical uniform distribution for this sample. We are able to train a model that is robust to produce simplified trajectories with varying (for example, different user-specified compression ratios) compression ratios by sampling m, as opposed to using a static compression ratio, as is the case with other seq2seq tasks.

Parallel-Attentive Mechanism. The output of the decoder must be a subset of the input of the encoder. Then, we can calculate the attention score of each point, and store that into a HashMap, so that the decoder can directly go to the HashMap to get it when inferring, and the time is $O(1)$. Next, the self-attention part is fully-parallel. Assume that the input length

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.

is n, we can directly create n threads to complete this step, enabling a fully parallel-attentive encoder-decoder.

C. Differentiable Layer

As a trajectory exhibits strong spatial dependence, to generate T', we sample its neighbor-aware locations p_{st} from the distribution $p(p_{st}|T)$. Besides, to avoid repeated sampling for the same location, we will label each sampled location during model training/inference. Inspired by [16], a simple workaround during training is to pass a weighted sum of all the vocabulary's (V) grid embeddings as input to the next step of the Compressor's decoder and to the corresponding step of the Constructor's encoder, using a peaked softmax function:

$$\overline{g}_t^c = \sum_i^{|V|} g(w_i) Softmax(u_t^c \tau) \tag{8}$$

where u_t^c denotes the unnormalized score in Eq. 6 for each position w_i, and $\tau \in (0, +\infty)$ is the temporal dependence. When τ decreases to 0, most sampling locations in Eq. 8 go to the most probable positions.

As a low-variance approach to sampling from categorical distributions, we employ the Gumbel Softmax (GS) [38] in this study. Taking a particular p_{st} sample from the softmax (Eq. 6) is equivalent to subtracting argmax from an independent noise sample ξ from the Gumbel distribution and adding it element-wise to the logits:

$$p_{st} \sim Softmax(u_t^c) \leftrightarrow p_{st} = arg\ max(u_t^c) + \xi \tag{9}$$

Therefore, we can modify Eq. 8 as:

$$\tilde{g}_t^c = \sum_i^{|V|} g(w_i) Softmax((u_t^c + \xi)\tau) \tag{10}$$

The above sampling strategy has been proved with quality guarantee [10], [16]. In this paper, the above sampling is just a single processing step and is not the main contribution. We utilize it due to its simple implementation and efficiency, as well as it has been successfully used in NLP tasks such as machine translation [24] and news abstraction [3], [10]. Take note of the fact that the mixtures of embeddings produced by the two relaxations do not correspond to the actual geolocations. Even though this makes it possible for the Compressor and the Constructor to communicate with each other using continuous values, making full use of the embedding space that is available, our ultimate goal is to restrict their communication to only real locations. Additionally, there is a difference between the test and training phases. Also, we proceed to use neighbor-based sampling to select the most neighboring trajectory points during the sampling phases. During the forward training pass, we discretize $\tilde{g}_t^c$ with Eq. 9 while we use Eq. 10 to calculate the gradients in the backward pass. Finally, a length vector and a transformation of the last hidden states of each bidirectional encoder are used to initialize the hidden states of each decoder.

$$h_0^c = \tanh\left(W_c\left[\overrightarrow{h_N^s}; \overleftarrow{h_1^s}; w_v M; \frac{M}{N}\right]\right) \tag{11}$$

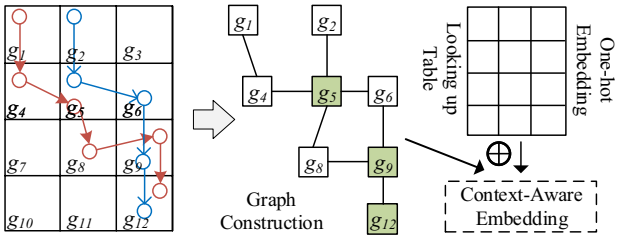


Fig. 6. Illustration of global mobility embedding

where W_c is a trainable hidden layer.

D. Constructor

The Constructor (shown in Fig. 4) is opposite to Compressor. It's encoder operates on the embeddings $g_1^c, ..., g_m^c$ of the locations $p_{s_1}, ..., p_{s_m}$ for the simplified trajectory.

Loss Functions. The Reconstruction Loss $L_R(p, \hat{p})$ is widely used in representation learning studies shown in Section II-C, which denotes the negative log-likelihood computed by the Constructor for the input trajectory $T = \langle p_1, ..., p_n \rangle$, where p_R denotes the sampling distribution.

$$L_R(p, \hat{p}) = -\sum_{i=1}^{n} log p_R(\hat{p}_i = p_i) \tag{12}$$

Note that, with the Embedding layer (see Sec. IV-A), The vocabulary V is formed by all of the grid cells, which are disjointed and equal in size. Then, we assign each position p into a unified cell and treat it as a token. That is, we adopt a one-hot vector for each p to express the uniqueness of its location in the feature matrix. In order to improve model efficiency, we do not anticipate that $L_R(p, hatp)$ will fall below 0. Instead, we anticipate that it will increase the likelihood of target points during reconstruction—a common speed-up training strategy in the community.

E. Complexity Analyses

The complexity of basic S3 approach is $O(n)$. Here, n denotes the length of a target trajectory. Specifically, the time is dominated by two components, i.e., Compressor and Constructor. As both components own the same Seq2Seq, we only need to analyze one of them, whose time relies on the Seq2Seq and Attention. We analyze them as follows. In Seq2Seq, the time cost for one original trajectory consists of (i) that of obtaining embeddings of points, which is $O(1)$, and (ii) that of obtaining the deep representation via a bi-RNN, which is $O(1)$. In Attention layer, the time cost for position weight computation is $O(n)$. Therefore, the time complexity of S3 is $O(1) \times O(n) = O(n)$. Note that the $O(n)$ time complexity maintains the current best time complexity as those of existing algorithms for the trajectory simplification task.

V. Context-aware Extension of S3

To enable preserving global mobility patterns in trajectories, we extend S3 in the context of a graph neural architecture. First, we construct a graph to model context-aware relations for trajectories. Second, we present GNN to learn the initial embedding for each trajectory.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Online Trajectory Simplification

Input: a streaming trajectory $T = \langle p_1, p_2, \cdots \rangle$ which is inputted in an online manner, online buffer size W, compression ratio δ, state cache $\mathcal{C}$
Output: compressed trajectory $T_o = \langle\rangle$

initialize state cache $\mathcal{C} = []$, current trajectory $T_c = []$
for *each* p_i **do**
  Call Eembedding Layer to obtain $p_i \rightarrow g_i$
  $T_c \leftarrow$ Concat (T_c, g_i)
  if *Length* (T_c)<W **then**
    $T_o, h_i^e \leftarrow$ Compressor (T_c)
    $T_o \leftarrow$ Differentiable Layer(T_o)
    $T_c \leftarrow$ Compare and Update (T_c, T_o)
    $\mathcal{C} \leftarrow$ Cache Update $(\mathcal{C}, T_o, h_i^e)$
  if *Length* $(T_c) \geq W$ **then**
    $T_o, h_{0:i}^e \leftarrow$ Compressor (T_c)
    $T_o \leftarrow$ Differentiable Layer(T_o)
    $T_c \leftarrow$ Compare and Update (T_c, T_o)
    $\mathcal{C} \leftarrow$ Cache Update $(\mathcal{C}, T_o, h_{0:i}^e)$
return T_o

A. Graph Construction

To capture the above global mobility dependence, a typical way of previous studies [12], [17] is to use prior road network or POI knowledge. However, such external information is not always available in real-life applications. In contrast, we utilize raw trajectories to construct the spatial graph to capture the global dependence through capturing the correlations among spatial locations in all trajectories. Specifically, we construct a context-aware spatial graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node in $\mathcal{G}$ denotes a grid cell, and edges indicate the connectivity between grid cells. As depicted in Fig. 6, an edge is created between grid cell g_i and grid cell g_j if a trajectory contains a consecutive snippet from $g_i \backslash g_j$ to $g_j \backslash g_i$. In high-sampling trajectories, some consecutive spatio-temporal points may be mapped to the same grid cell. In this case, we remove the self-loop edges in $\mathcal{G}$. The weight on edge $e_{i,j} \in \mathcal{E}$ is defined as the number of trajectories that contain the consecutive snippet $(g_i \backslash g_j \leftrightarrow g_j \backslash g_i)$. Furthermore, the semantic context of nodes can be expressed by a feature matrix. Note that, in this paper, we adopt a one-hot vector (the length equals to the number of grid cells) for each node g_i to express the uniqueness of its location in the feature matrix. We use A and X to denote adjacency matrix and feature matrix.

B. Context-Aware Graph Convolutions

GNNs are widely used in non-Euclidean space such as traffic networks and social networks to capture graph topological structure by aggregating neighbor features, and have achieved great success [17], [46]. As we construct context-aware spatial graph $\mathcal{G}$, we employ GCN (one type of GNNs) on $\mathcal{G}$ to learn the hierarchical embeddings of trajectories. Specifically, we perform graph convolution operation on $\mathcal{G}$ to obtain context-aware trajectory embeddings. Specifically, multi-layer spatial convolution network performs layer-wise propagation:

$$\mathbf{H}_l^{i+1} = \mathit{ReLU}(\tilde{\mathbf{D}}_l^{-\frac{1}{2}} \tilde{\mathbf{A}}_l \tilde{\mathbf{D}}_l^{-\frac{1}{2}} \mathbf{H}_l^{(i)} \mathbf{W}_l^{(i)}), \tag{13}$$

where $\mathbf{W}_l^{(i)}$ is a layer-specific trainable matrix, $\tilde{\mathbf{A}}_l = \mathbf{A} + \mathbf{I}$, and $\tilde{\mathbf{A}}_{l_{ii}} = \sum_j \tilde{\mathbf{A}}_{l_{ii}}$. $\mathbf{H}_l^{(0)} = \mathbf{X}_l$, where $\mathbf{X}_l$ is feature matrix of $\mathcal{G}$. $\mathbf{H}_l^{(i)} \in \mathbb{R}^{n \times d}$ is output of the i-th layer (i.e., the embeddings of all cells), as d is the embedding dimension.

C. Context-aware Error Distance (CED)

For quantitative analysis, we present a context-aware error distance to evaluate the spatio-context information loss between a raw trajectory T and its simplified version T'. Inspired by the linear combination approach [6], [49], [50], [57], we define CED distance between T and T' as below.

$$\mathrm{CED}\left(T, T'\right) = \frac{1}{2}\left(\frac{\sum_{p_i \in T} r\left(p_i, T'\right)}{|T|} + \frac{\sum_{p_{sj} \in T'} r\left(p_j, T\right)}{|T'|}\right) \tag{14}$$

where $r(p, T) = \max_{p_i \in T}\{SC\left(p, p_i\right)\}$ and $SC\left(p_i, p_j\right) = S\left(p_i, p_j\right) + C\left(p_i, p_j\right)$. Here, $S(p_i, p_j)$ denotes the normalized L_2 spatial distance between p_i and p_j, i.e., the spatial difference between two positions; $C(p_i, p_j)$ denotes the normalized cosine semantic distance between p_i and p_j, i.e., the semantic difference between two positions. Note that, following Hausdorff distance [2] that is widely used to compute the inter-trajectory distances, we also use the max function. In that way, we evaluate the maximum mismatch between two point sets, i.e., the maximum mismatch between T and $\hat{T}$.

Superiority of CED. PED, SED, and DED capture spatial information such as spatial distance, speed, and direction. However, trajectories contain not only spatial information, but also semantic information (e.g., traveling POIs). Only using spatial features for simplification may cause semantic information loss, which may hurt semantic-aware analyses such as semantic trajectory similarity queries [6]. CED can capture both spatial-aware and semantic-aware features via its linear combination of spatial distance and semantic distance between two trajectories. Fig. 1 demonstrates its superiority.

Complexity analyses. The time complexity of graph construction is $O(|\mathcal{T}|)$ as we need to scan all trajectories to form the graph $\mathcal{G}$. According to Eq. 13, the time complexity of spatial GCN block is $O(|\mathcal{E}|d)$ for GCN operation, for every iteration. $|\mathcal{E}|$ denotes #edges and d is embedding dimension.

VI. Online Extension of S3

In online settings, trajectory points that have been dropped will no longer be accessible, where the trajectory that needs to be simplified is arriving point by point. Applications like remote sensing, where the sensors must work within a limited storage budget, network bandwidth, and energy, frequently employ this mode. However, it is non-trivial to directly deploy S3 into online settings as the biggest difference between offline and online settings is the high-efficiency requirements of the latter case. According to the complexity analyses of offline S3 approach, it is inefficient to directly deploy S3 in online settings. To this end, we propose an online extension of S3, which features two modifications. First, when each point passes the Compressor, we cache its embedding into a hash table, which can be directly connected with next point

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.

TABLE III
STATICS OF DATASETS USED

Attributes	GeoLife	T-drive
# of trajectories	17,621	10,359
Total of # Points	24,876,978	17,740,902
Ave. trajectory length	1,412	1,713
Sampling rate	$1 \sim 5$s	$1 \sim 177$s
Ave. distance	9.96m	623m

to consist of the final hidden state. Thus, we can reduce the complexity of Compressor from $O(n)$ to $O(1)$. Second, in terms of self-attention layer, we also adopt similar idea to persist pre-computed attention scores for each arriving point.

Algorithm 1 shows the pseudo-code of our online S3. For each arriving point p_i, it first embeds this point into its hidden vector representation through the looking up table in the embedding layer (line 3). Next, it concats this single point embedding into its previous sequence embedding directly (line 4). If the embedding length exceeds the buffer size W, it performs trajectory simplification using the Compressor and Differential Layer (lines 11–14); otherwise, it proceeds to initialize the embedded trajectories (lines 6–9).

VII. EXPERIMENTS

A. Experimental Settings

Datasets. We use two kinds of real datasets to determine whether an algorithm is adaptable, considering the characteristics of a trajectory may significantly differ depending on the mode of transportation of moving objects.

- **GeoLife**[1] contains the traces of each user for a period of more than three years, including multiple kinds of transportation modes (walking, driving, and public transportation). The GPS point is collected periodically, and 91% of the trajectories are sampled every 5 seconds.
- **T-drive**[2] contains the trajectories of 10,357 taxis in Beijing, China, during a period of 7 days with sampling rates varying between 1 and 177 seconds.

Baselines and Evaluation metrics. We review the trajectory simplification serveries [31], [62] thoroughly so that all representative and popular min-ϵ oriented trajectory simplification methods are included for comparison. Overall, we identify seven compared methods in terms of both non-learning and learning based categories. (i) For non-learning-based methods, we choose (1) Top-Down [19], (2) Bottom-Up [36], (3) the state-of-the-art Error-Search [37] for the offline mode, (4) STTrace [47], (5) SQUISH [43], and (6) SQUISH-E [44] for the online mode. (ii) For learning-based methods, (7) RLTS [54], [55] is the one and the only one that utilizes AI techniques (i.e., reinforcement learning) to achieve state-of-the-art data-driven trajectory simplification for offline and online modes, which is also an approximate algorithm. Note that, we use PED, SED, DAD, and CED for algorithm quality evaluation, and use running time for algorithm efficiency evaluation. S3 can process trajectories in groups like Trajstore [8] due to its batch processing.

[1]https://research.microsoft.com/en-us/projects

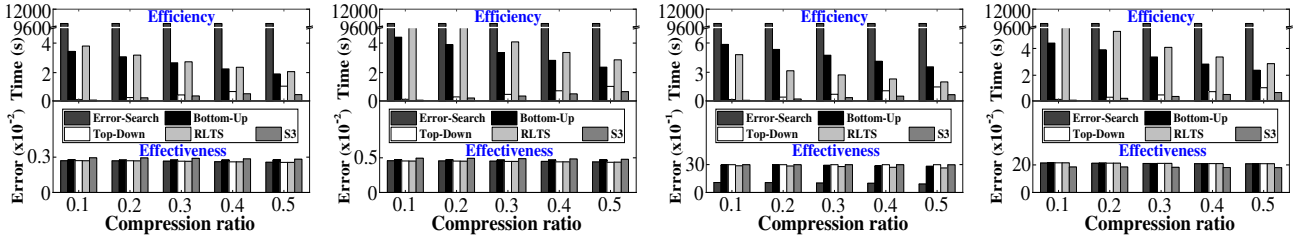
[2]http://www.geolink.pt/ecmlpkdd2015-challenge/dataset.html

Parameter settings. The length of each spatial grid cell is set to 100 meters by default. Since the S3 model's embedding performance is superior to that of the LSTM, it uses a GRU with three layers as its fundamental computational unit. In addition, we clip the gradients using the maximal gradient norm constraint, which is set to 5 in our experiments. In addition, we train the model using Adam stochastic gradient descent with 500 iterations and an initial learning rate of 0.0001. Take note of the fact that after repeating each comparison method twenty times, we present the average performance. The units of SED, PED, and DAD are 10m, 10m, and 1 radian, respectively. For different compression ratios under the same dataset, there is no need to train S3 for each compression ratio. In S3, the length of the encoder of Compressor and the length of the decoder of Constructor refer to the length of the input trajectory; while the length of the decoder of the Compressor and the length of the encoder of the Constructor refer to the length of the simplified trajectory. Note that, these length parameters can be flexibly set to any value. We train S3 under the default ratio (i.e., 0.3). Once S3 is well-trained for model inference, we adjust the length of the decoder of the Compressor, and the length of the encoder of the Constructor, for different ratios. Such a process is the same as general Seq2Seq training [24]. For different datasets, we train S3, respectively, as different datasets may exhibit different mobility features that are required to be learned to guide the model training. Python and PyTorch were used in the framework's implementation. Last but not least, all of the experiments were carried out on a server equipped with a GeForce GTX-2080 Ti 11G GPU, an Intel Silver 4210R processor running at 2.40 GHz, and 16 GB of RAM.

B. Offline Trajectory Simplification Study

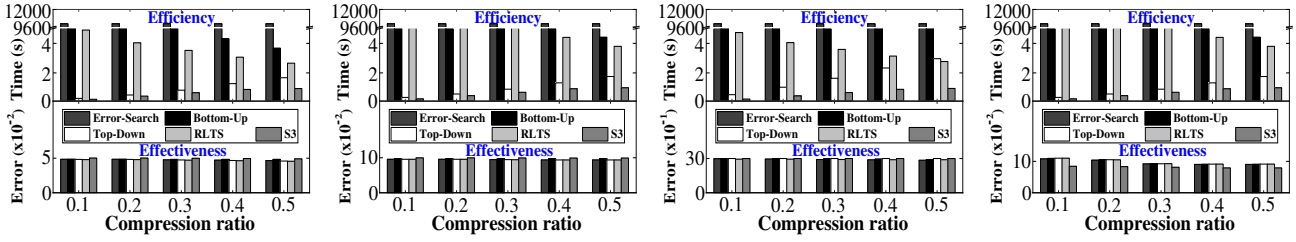
Offline efficiency-effectiveness trade-offs. Figs. 7 and 8 show the offline efficiency-effectiveness trade-offs of different algorithms using GeoLife and T-drive datasets, respectively. Note that, the upper side of a figure represents efficiency (i.e., running time), and the lower side of a figure represents effectiveness (i.e., simplification error). For a fair comparison, following previous experimental studies [54], [55], we randomly select 1,000 trajectories and fix each $|T|$ at 25 ($\text{x}10^2$), and then, we report the running time and simplification error of each algorithm under each error metric to compress each trajectory when varying compression ratio from 0.1 to 0.5. Here, the running time is computed as the average amount of time it takes the CPU to process one trajectory simplification. Overall, we have the following observations. First, although Error-Search achieves the best effectiveness (i.e., smallest simplification errors) on existing error distances including PED, SED, and DAD, it runs too slowly (i.e., it runs more than 10,000 seconds), which is very inefficient for real-life applications. Second, S3 runs much faster than classic Top-Down and Bottom-Up methods and the state-of-the-art learning-based RLTS method up to $10\times$, but has a very slight effectiveness drop (i.e., an increase of error is no more than 0.001 on PED, 0.003 on SED, and 0.01 on DAD, while error $\in [0, +\infty)$) and has a slight effectiveness

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.



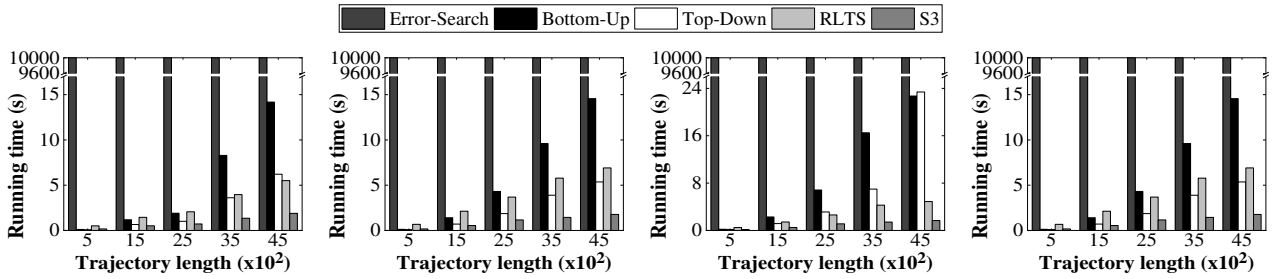
(a) PED Error/GeoLife (b) SED Error/GeoLife (c) DAD Error/GeoLife (d) CED Error/GeoLife

Fig. 7. Offline settings: Efficiency-effectiveness trade-offs comparison on GeoLife vs. Compression ratio



(a) PED Error/T-drive (b) SED Error/T-drive (c) DAD Error/T-drive (d) CED Error/T-drive

Fig. 8. Offline settings: Efficiency-effectiveness trade-offs comparison on T-drive vs. Compression ratio



(a) PED Error/T-drive (b) SED Error/T-drive (c) DAD Error/T-drive (d) CED Error/T-drive

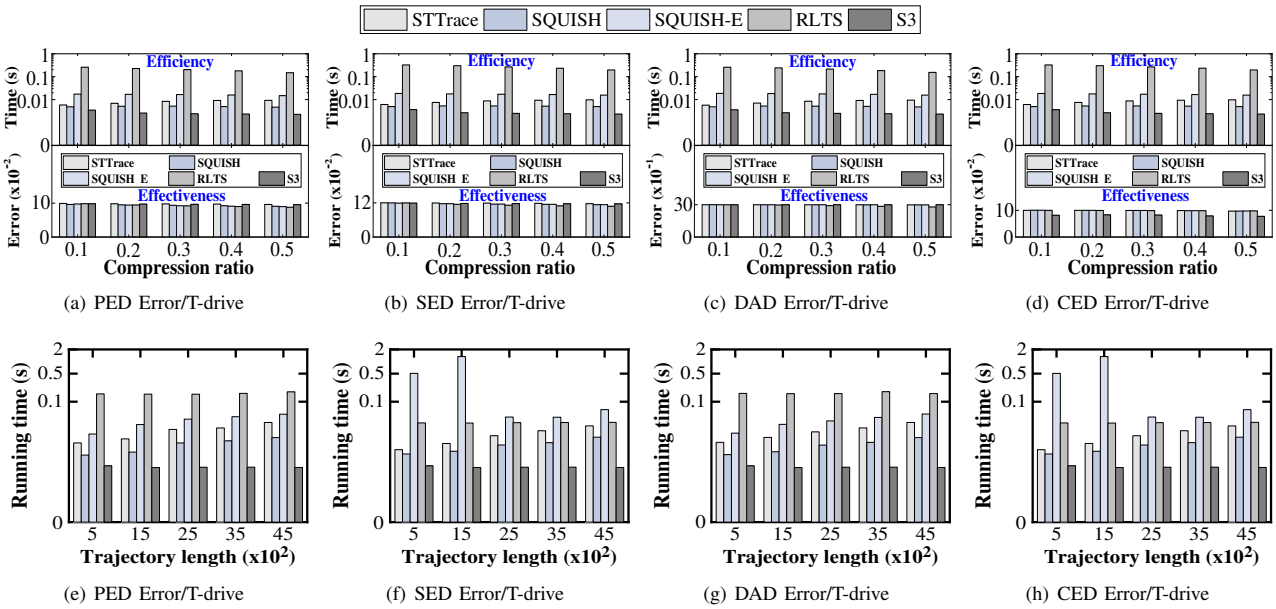
Fig. 9. Offline settings: simplification efficiency comparison on T-drive vs. Trajectory length

improvement on our designed CED error distance. This is owing to S3's efficient framework that decoupled from massive error distance computations, indicating the potentiality of S3 for large-scale trajectory simplification. Third, S3 acquires slight effectiveness drops on PED, SED, and DAD metrics, while achieving the best effectiveness on the CED metric. This is because each proposed trajectory simplification algorithm is typically optimized for one specific error distance. S3 performs best on our designed CED error distance that considers both spatial and semantic features, while naturally performing slightly worse on existing PED, SED, and DAD distances that respectively focus on distance, speed, and direction features.

Taking PED on GeoLife as example (i.e., Fig. 7(a)), under compression ratio of 0.5, S3 needs 0.51s for per trajectory simplification: ten thousands times faster than Error-Search (10,000+s), 3.8 times faster than Bottom-Up (about 1.91s), 2.1 times faster than Top-Down (about 1.08s), and 4.0 times faster than RLTS (about 2.06s), at cost of very slight error increase: 0.0007 error increase than Error-Search, 0.0003 error increase than Bottom-Up, 0.0008 error increase than Top-Down, and 0.0008 error increase than RLTS. Here, error $\in [0, +\infty)$.

Following previous studies [54], [55], we fix the compression ratio as 0.1, and we study the simplification efficiency by varying trajectory length from 5 (x10^2) to 45 (x10^2). Fig. 9 plots the results. First, as the length of an algorithm increases, so does its running time. This is straightforward, because more points in a trajectory need to be processed in order to simplify. It is important to note that due to its data-driven and lightweight processing, S3 has the most stable performance. The second observation is that S3 performs better than all learning-based and non-learning methods, particularly when trajectories are longer, indicating high efficiency. For example, for trajectories with about 35 (x10^2) points under DAD error distance (i.e., Fig. 9(c)), S3 takes less than 1.5s for per trajectory simplification, which is 10 thousand times faster than Error-Search (10,000+s), 10 times faster than Bottom-Up (about 15s), 2 times faster than Top-Down (about 3s), and 4 times faster than RLTS (about 6s). Such high efficiency enables S3 to support efficient downstream trajectory analysis, as trajectory compression is a pre-processing step in the trajectory data management pipeline. Other observations are summarized: (i) Error-Search requires $O(n^2 logn)$ complexity and runs timeout (10,000+ seconds). (ii) The learning-based RLTS cannot run faster than classic algorithms, as reinforcement learning relies on decision interactions to make many decisions, introducing additional computational costs.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.



(a) PED Error/T-drive (b) SED Error/T-drive (c) DAD Error/T-drive (d) CED Error/T-drive

(e) PED Error/T-drive (f) SED Error/T-drive (g) DAD Error/T-drive (h) CED Error/T-drive

Fig. 10. Online performance comparison,(a)–(d): Efficiency-effectiveness trade-offs vs. Compression ratio; (e)–(h): Efficiency vs. Trajectory length

C. Online Trajectory Simplification Study

We also compare S3 with existing online algorithms. In particular, we select 1,000 trajectories at random from T-drive each with total 25 (x10^2) points, and then, we input these trajectory points one by one in a streaming manner. Figs. 10(a)–10(d) fix the streaming length at 25 (x10^2) points, and report the processing time (for single point) and simplification errors by varying compression ratios. Figs. 10(e)–10(h) depict the compression time for single point by varying input streaming trajectory lengths, from 5 (x10^2) to 25 (x10^2) points. Here, we only use the T-drive. First, S3 shows the best and 10×–100× efficiency gains, compared with baselines. In terms of effectiveness, i) S3 acquires the smallest errors on CED as S3 is typically optimized for CED error distance; and ii) even though, S3 achieves the medium effectiveness on other error distances (PED, SED, and DAD), which exhibits a slight error increase of less than 0.001 (error $\in [0, +\infty)$) over the most accurate method exists.

D. Case Study

To demonstrate that the effectiveness gap between S3 and existing methods has no impact, we conduct case studies. Specifically, we evaluate the quality of the trajectory dataset after simplification processing. That is, we evaluate the effect of trajectory simplification on downstream trajectory analysis tasks, confirming the resulted errors (i.e., data loss) of both S3 and baselines are acceptable. Following previous studies [54], [62], we select the top-k trajectory similarity query as the evaluation task, since similarity evaluation is the most essential task for trajectory analytics, which is most suitable to reflect the impact of raw data changes on analysis results. Specifically, we utilize top-k similarity search for validation, where we use two widely used trajectory similarity metrics, the point-based DTW [61] and the shape-based Frechet [40]. Due to space limitation, we use DTW to verify error effect of algorithms using PED and SED, while use Frechet to evaluate error effect of algorithms using DAD and CED.

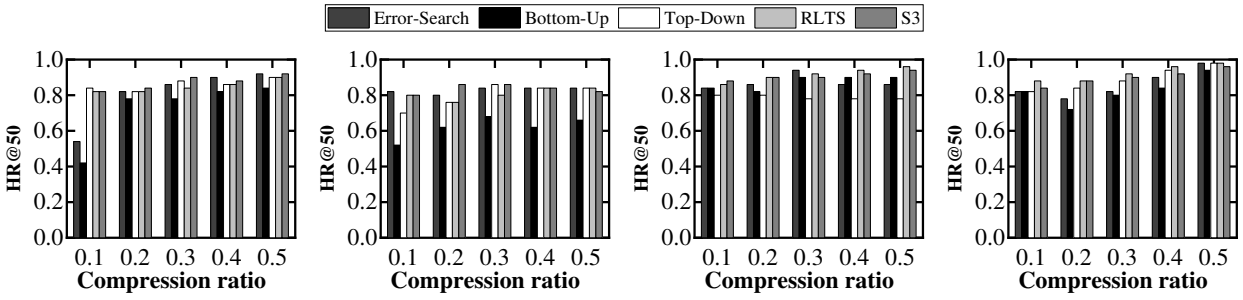
We adopt HR@50 as a performance metric. HR@50 denotes the top-50 hitting ratio that captures the degree of overlap between top-50 search results returned from a raw trajectory dataset and that returned from a simplified trajectory dataset. The HR@50 approaches to 1, the less the error effect on downstream tasks. Fig. 11 plots the results. Here, we use T-drive dataset due to similar observations on GeoLife dataset. As observed, under the same ratios, S3 yields nearly the same or better HR@50 (i.e., 0.80–0.96) to the baselines that show 0.42–0.98 HR@50. This demonstrates that, although S3 cannot defeat all baselines on effectiveness, the quality gap is totally acceptable, as S3 and baselines show the equivalent effect on the downstream trajectory analysis tasks.

E. Model Analysis

We first evaluate the training cost of S3, which is important for deployment. We compare S3 with RLTS, as RLTS is the one and the only that utilizes AI techniques. The training times and CED errors of RLTS and S3 on T-drive are shown in Fig. 12. Due to the large-scale computation space of reinforcement learning, we find that RLTS takes several hours to train a model. In contrast, S3 only takes several minutes for model training, which significantly improves the usability in real-life applications. Besides, S3 achieves comparable quality with RLTS, due to its data-driven ability.

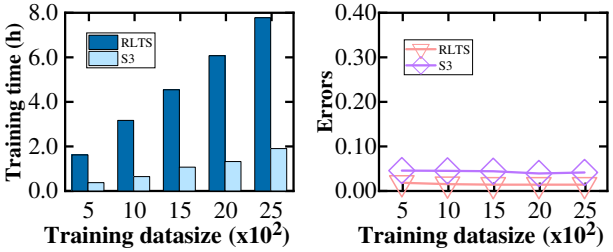
We then evaluate the memory usage of S3. Here, we use the CED metric as an illustration due to similar observations on other error distances. Fig. 13 shows the running memory usage of S3 and RTLS by varying the compression ratio on GeoLife and T-drive datasets. As observed, S3 occupies nearly half fewer memory usage than RLTS, as S3 is a very light representation learning based unsupervised framework. In contrast, RLTS relies on reinforcement learning and has to make massive decision computations.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.



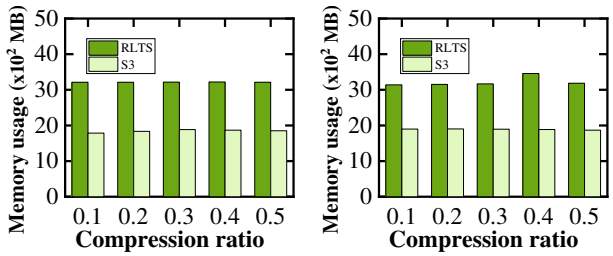
(a) PED Error/DTW Metric (b) SED Error/DTW Metric (c) DAD Error/Frechet Metric (d) CED Error/Frechet Metric

Fig. 11. Case studies: The impacts of simplification error on top-k trajectory similarity queries



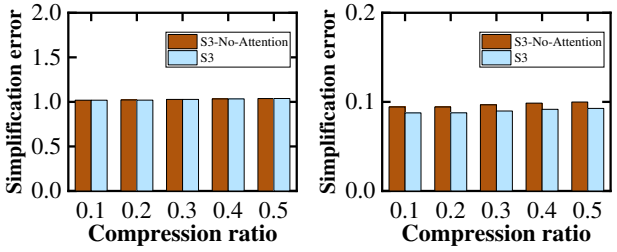
(a) Training cost on T-drive (b) Training error on T-drive

Fig. 12. Training cost and error evaluation



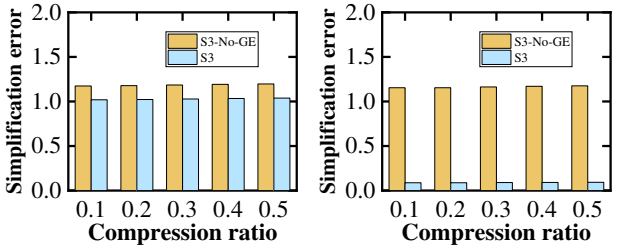
(a) Memory usage on GeoLife (b) Memory usage on T-drive

Fig. 13. Model memory usage vs. Compression ratio



(a) GeoLife (b) T-drive

Fig. 14. Performance of S3 vs. Attention



(a) GeoLife (b) T-drive

Fig. 15. Performance of S3 vs. Context embedding

We finally conduct an ablation study. Here, we only use the CED measure as an illustration due to limited space. To verify the effect of self-attention, we train S3 with and without self-attention, respectively. Fig. 14 shows that S3 using self-attention achieves slightly better effectiveness. To evaluate the effect of context-aware graph embedding, we remove it, resulting in another variant of S3 called S3-No-GE. Fig. 15 shows that the use of context embedding yields higher quality.

F. Experimental Summary

Overall, S3 exhibits light model deployment, much better model efficiency, and comparable effectiveness. (i) In terms of model training, S3 acquires 4× convergence rate gains than the existing methods; (ii) In terms of model efficiency, S3 achieves up to 10× speed-up gains in offline settings and 100× speed-up gains in online settings; (iii) In terms of model effectiveness, although S3 trades effectiveness for efficiency, there is a slight effectiveness drop, i.e., less than 0.01 error-increases (error $\in [0, +\infty)$). Second, when evaluating the impact of slight effectiveness sacrifice via downstream trajectory top-k queries, S3 shows 0.80–96 hitting ratios, which is nearly same or better than baselines that show 0.42–0.98 hitting ratios.

VIII. Conclusions

Existing trajectory simplification approaches mostly rely on human rules or pre-defined parameters, and they are closely coupled with error metrics, resulting in low versatility, intensive computation, and inefficiency challenges. Motivated by these, we propose a lightweight representation leaning based framework, abbreviated S3, to perform self-supervised trajectory simplification in an end-to-end manner. Compared with existing algorithms, S3 is data-driven, and more importantly, it achieves very high efficiency in terms of both offline and online scenarios, while exhibiting comparable quality. Besides, both model training and model inference are lightweight. A semantic-based deep trajectory simplification, like the road network and POIs, is appealing in the future.

IX. Acknowledgment

This work was supported in part by the NSFC under Grants No. (62025206, 61972338, and 62102351), Hangzhou Technology Bureau AI Research Project under Grants No. 2022AIZD0116, and Ningbo Science and Technology Special Innovation Projects with Grant Nos. 2022Z095 and 2021Z019. Yunjun Gao is the corresponding author of the work.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] DiDi Brain. https://www.didiglobal.com/science/brain, 2018.

[2] S. Atev, G. Miller, and N. P. Papanikolopoulos. Clustering of vehicle trajectories. *TITS*, 11(3):647–657, 2010.

[3] C. Baziotis, I. Androutsopoulos, I. Konstas, and A. Potamianos. Seq^3: Differentiable sequence-to-sequence-to-sequence autoencoder for unsupervised abstractive sentence compression. In *NAACL-HLT*, pages 673–681. Association for Computational Linguistics, 2019.

[4] R. Bellman. On the approximation of curves by line segments using dynamic programming. *Communications of the ACM*, 4(6):284, 1961.

[5] H. Cao and O. Wolfson. Nonmaterialized motion information in transport networks. In *International Conference on Database Theory*, pages 173–188, 2005.

[6] L. Chen, S. Shang, C. S. Jensen, B. Yao, and P. Kalnis. Parallel semantic trajectory similarity join. In *ICDE*, pages 997–1008. IEEE, 2020.

[7] Y. Chen, X. Li, G. Cong, Z. Bao, C. Long, Y. Liu, A. K. Chandran, and R. Ellison. Robust road network representation learning: When traffic patterns meet traveling semantics. In *CIKM*, pages 211–220, 2021.

[8] P. Cudre-Mauroux, E. Wu, and S. Madden. Trajstore: An adaptive storage system for very large trajectory data sets. In *ICDE*, pages 109–120, 2010.

[9] D. H. Douglas and T. K. Peucker. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartographica*, 10(2):112–122, 1973.

[10] A. Fan, D. Grangier, and M. Auli. Controllable abstractive summarization. In *Proceedings of the 2nd Workshop on Neural Machine Translation and Generation, NMT@ACL 2018, Melbourne, Australia, July 20, 2018*, pages 45–54. Association for Computational Linguistics, 2018.

[11] Z. Fang, Y. Du, L. Chen, Y. Hu, Y. Gao, and G. Chen. E^{2}dtc: An end to end deep trajectory clustering framework via self-training. In *ICDE*, pages 696–707, 2021.

[12] Z. Fang, Y. Du, X. Zhu, D. Hu, L. Chen, Y. Gao, and C. S. Jensen. Spatio-temporal trajectory similarity learning in road networks. In *SIGKDD*, pages 347–356, 2022.

[13] Z. Fang, L. Pan, L. Chen, Y. Du, and Y. Gao. MDTP: A multi-source deep traffic prediction framework over spatio-temporal trajectory data. *Proc. VLDB Endow.*, 14(8):1289–1297, 2021.

[14] C. Gao, Z. Zhang, C. Huang, H. Yin, Q. Yang, and J. Shao. Semantic trajectory representation and retrieval via hierarchical embedding. *Information Sciences*, 538:176–192, 2020.

[15] C. Gao, Y. Zhao, R. Wu, Q. Yang, and J. Shao. Semantic trajectory compression via multi-resolution synchronization-based clustering. *Knowledge-Based Systems*, 174:177–193, 2019.

[16] K. Goyal, C. Dyer, and T. Berg-Kirkpatrick. Differentiable scheduled sampling for credit assignment. In R. Barzilay and M. Kan, editors, *ACL*, pages 366–371. Association for Computational Linguistics, 2017.

[17] P. Han, J. Wang, D. Yao, S. Shang, and X. Zhang. A graph-based approach for trajectory similarity computation in spatial networks. In *SIGKDD*, pages 556–564, 2021.

[18] Y. Han, W. Sun, and B. Zheng. Compress: A comprehensive framework of trajectory compression in road networks. *TODS*, 42(2):1–49, 2017.

[19] J. E. Hershberger and J. Snoeyink. Speeding up the douglas-peucker line-simplification algorithm. 1992.

[20] R. Jiang, D. Yin, Z. Wang, Y. Wang, J. Deng, H. Liu, Z. Cai, J. Deng, X. Song, and R. Shibasaki. Dl-traff: Survey and benchmark of deep learning models for urban traffic prediction. In *CIKM*, pages 4515–4525, 2021.

[21] B. Ke, J. Shao, and D. Zhang. An efficient online approach for direction-preserving trajectory simplification with interval bounds. In *MDM*, pages 50–55, 2017.

[22] B. Ke, J. Shao, Y. Zhang, D. Zhang, and Y. Yang. An online approach for direction-based trajectory compression with error bound guarantee. In *Asia-Pacific Web Conference*, pages 79–91, 2016.

[23] G. Kellaris, N. Pelekis, and Y. Theodoridis. Map-matched trajectory compression. *Journal of Systems and Software*, 86(6):1566–1579, 2013.

[24] Y. Kikuchi, G. Neubig, R. Sasano, H. Takamura, and M. Okumura. Controlling output length in neural encoder-decoders. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, EMNLP 2016, Austin, Texas, USA, November 1-4, 2016*, pages 1328–1338. The Association for Computational Linguistics, 2016.

[25] R. Lange, F. Dürr, and K. Rothermel. Efficient real-time trajectory tracking. *The VLDB Journal*, 20(5):671–694, 2011.

[26] T. Li, L. Chen, C. S. Jensen, and T. B. Pedersen. Trace: Real-time compression of streaming trajectories in road networks. *VLDB*, 14(7):1175–1187, 2021.

[27] T. Li, R. Huang, L. Chen, C. S. Jensen, and T. B. Pedersen. Compression of uncertain trajectories in road networks. *VLDB*, 13(7):1050–1063, 2020.

[28] X. Li, K. Zhao, G. Cong, C. S. Jensen, and W. Wei. Deep representation learning for trajectory similarity computation. In *ICDE*, pages 617–628, 2018.

[29] C. Lin, C. Hung, and P. Lei. A velocity-preserving trajectory simplification approach. In *Conference on Technologies and Applications of Artificial Intelligence, TAAI 2016, Hsinchu, Taiwan, November 25-27, 2016*, pages 58–65. IEEE, 2016.

[30] X. Lin, J. Jiang, S. Ma, Y. Zuo, and C. Hu. One-pass trajectory simplification using the synchronous euclidean distance. *The VLDB Journal*, 28(6):897–921, 2019.

[31] X. Lin, S. Ma, J. Jiang, Y. Hou, and T. Wo. Error bounded line simplification algorithms for trajectory compression: An experimental evaluation. *TODS*, 46(3):1–44, 2021.

[32] X. Lin, S. Ma, H. Zhang, T. Wo, and J. Huai. One-pass error bounded trajectory simplification. *VLDB*, 10(7):841–852, 2017.

[33] J. Liu, K. Zhao, P. Sommer, S. Shang, B. Kusy, and R. Jurdak. Bounded quadrant system: Error-bounded trajectory compression on the go. In *ICDE*, pages 987–998, 2015.

[34] J. Liu, K. Zhao, P. Sommer, S. Shang, B. Kusy, J.-G. Lee, and R. Jurdak. A novel framework for online amnesic trajectory compression in resource-constrained environments. *IEEE Transactions on knowledge and data engineering*, 28(11):2827–2841, 2016.

[35] M. Liu, G. He, and Y. Long. A semantics-based trajectory segmentation simplification method. *Journal of Geovisualization and Spatial Analysis*, 5(2):1–15, 2021.

[36] C. Long, R. C.-W. Wong, and H. Jagadish. Direction-preserving trajectory simplification. *VLDB*, 6(10):949–960, 2013.

[37] C. Long, R. C.-W. Wong, and H. Jagadish. Trajectory simplification: On minimizing the direction-based error. *VLDB*, 8(1):49–60, 2014.

[38] J. Mallinson, R. Sennrich, and M. Lapata. Sentence compression for arbitrary languages via multilingual pivoting. In *EMNLP*, pages 2453–2464, 2018.

[39] J. Mao, J. Liu, C. Jin, and A. Zhou. Feature grouping-based trajectory outlier detection over distributed streams. *ACM Trans. Intell. Syst. Technol.*, 12(2):22:1–22:23, 2021.

[40] d. B. Mark, C. Otfried, v. K. Marc, and O. Mark. *Computational geometry algorithms and applications*. Spinger, 2008.

[41] J. D. Mazimpaka and S. Timpf. Trajectory data mining: A review of methods and applications. *Journal of spatial information science*, 2016(13):61–99, 2016.

[42] N. Meratnia et al. Spatiotemporal compression techniques for moving point objects. In *EDBT*, pages 765–782, 2004.

[43] J. Muckell, J.-H. Hwang, V. Patil, C. T. Lawson, F. Ping, and S. Ravi. Squish: an online approach for gps trajectory compression. In *COM.Geo*, pages 1–8, 2011.

[44] J. Muckell, P. W. Olsen, J.-H. Hwang, C. T. Lawson, and S. Ravi. Compression of trajectory data: a comprehensive evaluation and new approach. *GeoInformatica*, 18(3):435–460, 2014.

[45] A. Nibali and Z. He. Trajic: An effective compression system for trajectory data. *TKDE*, 27(11):3138–3151, 2015.

[46] L. Oneto, N. Navarin, B. Biggio, F. Errica, A. Micheli, F. Scarselli, M. Bianchini, L. Demetrio, P. Bongini, A. Tacchella, and A. Sperduti. Towards learning trustworthily, automatically, and with guarantees on graphs: An overview. *Neurocomputing*, 493:217–243, 2022.

[47] M. Potamias, K. Patroumpas, and T. Sellis. Sampling trajectory streams with spatiotemporal criteria. In *SSDBM*, pages 275–284, 2006.

[48] F. Schmid, K.-F. Richter, and P. Laube. Semantic trajectory compression. In *International Symposium on Spatial and Temporal Databases*, pages 411–416, 2009.

[49] S. Shang, L. Chen, Z. Wei, C. S. Jensen, K. Zheng, and P. Kalnis. Trajectory similarity join in spatial networks. *Proc. VLDB Endow.*, 10(11):1178–1189, 2017.

[50] S. Shang, L. Chen, Z. Wei, C. S. Jensen, K. Zheng, and P. Kalnis. Parallel trajectory similarity joins in spatial networks. *VLDB J.*, 27(3):395–420, 2018.

[51] R. Song, W. Sun, B. Zheng, and Y. Zheng. Press: A novel framework of trajectory compression in road networks. *VLDB*, 7(9):661–672, 2014.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.

[52] G. Trajcevski, H. Cao, P. Scheuermanny, O. Wolfsonz, and D. Vaccaro. On-line data reduction and the quality of history in moving objects databases. In *MobiDE*, pages 19–26, 2006.

[53] S. Wang, J. Cao, and P. Yu. Deep learning for spatio-temporal data mining: A survey. *TKDE*, 2020.

[54] Z. Wang, C. Long, and G. Cong. Trajectory simplification with reinforcement learning. In *ICDE*, pages 684–695, 2021.

[55] Z. Wang, C. Long, G. Cong, and Q. Zhang. Error-bounded online trajectory simplification with multi-agent reinforcement learning. In *SIGKDD*, pages 1758–1768, 2021.

[56] C. M. Williams. Bounded straight-line approximation of digitized planar curves and lines. *Computer Graphics and Image Processing*, 16(4):370–381, 1981.

[57] C. Yang, L. Chen, H. Wang, and S. Shang. Towards efficient selection of activity trajectories based on diversity and coverage. In *AAAI*, pages 689–696. AAAI Press, 2021.

[58] M. Yang, X. Yan, X. Zhang, and X. Li. Constrained trajectory simplification with speed preservation. *Cartography and Geographic Information Science*, 47(2):110–124, 2020.

[59] P. Yang, H. Wang, Y. Zhang, L. Qin, W. Zhang, and X. Lin. T3s: effective representation learning for trajectory similarity computation. In *ICDE*, pages 2183–2188, 2021.

[60] D. Yao, C. Zhang, Z. Zhu, J. Huang, and J. Bi. Trajectory clustering via deep representation learning. In *IJCNN*, pages 3880–3887, 2017.

[61] B. Yi, H. V. Jagadish, and C. Faloutsos. Efficient retrieval of similar time sequences under time warping. In *ICDE*, pages 201–208, 1998.

[62] D. Zhang, M. Ding, D. Yang, Y. Liu, J. Fan, and H. T. Shen. Trajectory simplification: an experimental study and quality analysis. *VLDB* 11(9):934–946, 2018.

[63] Y. Zhang, A. Liu, G. Liu, Z. Li, and Q. Li. Deep representation learning of activity trajectory similarity computation. In *2019 IEEE International Conference on Web Services (ICWS)*, pages 312–319. IEEE, 2019.

[64] Y. Zheng. Trajectory data mining: an overview. *TIST*, 6(3):1–41, 2015.

[65] Y. Zheng, L. Capra, O. Wolfson, and H. Yang. Urban computing: concepts, methodologies, and applications. *TIST*, 5(3):1–55, 2014.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 09,2026 at 06:05:35 UTC from IEEE Xplore. Restrictions apply.